

HIERARCHICAL MULTI-AGENT COLLABORATIVE OPTIMIZATION FOR ACADEMIC POSTER GENERATION

Li Jiangtao

School of Computer Science, Fudan University

Student ID: 23307130167

23307130167@m.fudan.edu.com

ABSTRACT

Academic posters remain a primary medium for communicating research at conferences, yet manual design is time-consuming and existing automation tools often produce posters with fragmented logic, poor figure readability, and rigid iteration schedules. We present **PosterAgent**, a **training-free hierarchical multi-agent framework** that transforms PDF research papers into publication-quality academic posters without fine-tuning any layout or vision model. Our system introduces: (1) a **logic-first content pipeline** that extracts a cross-section *LogicPlan* and enforces semantic dependencies between Methods, Results, and Discussion; (2) a **semantically assigned three-column academic layout (25%/50%/25%)** coupled with **content-level column balancing** that adjusts text rather than distorting typography; and (3) a **paper-native figure pipeline** that renders, role-crops, and inserts PDF figures at panel boundaries to avoid mid-panel clipping and illegibly small visuals. A **multi-metric adaptive controller** fuses semantic completeness, layout balance, and image–text match scores to terminate refinement early when quality thresholds are met. On a challenging *Nature Methods* paper (GRASP, protein complex structure prediction), our system improves the composite quality score from **0.53 to 0.86** within three iterations. Source code and experiment artifacts are available at [GitHub.com/Ljt-lj/Hierarchical-Multi-Agent-Collaborative-Optimization-for-Academic-Poster-Generation](https://github.com/Ljt-lj/Hierarchical-Multi-Agent-Collaborative-Optimization-for-Academic-Poster-Generation).

1 INTRODUCTION

Conference posters condense long-form papers into a single visual narrative. Unlike slides or short summaries, posters must simultaneously satisfy *information completeness*, *logical coherence*, *visual hierarchy*, and *spatial efficiency* within a fixed canvas (typically 36×24 inches). Researchers spend hours manually selecting figures, writing bullet points, and balancing columns—tasks that are repetitive yet highly structured.

Recent work has explored automating poster generation with large language models (LLMs). **Paper2Poster** decomposes the task into Parser–Planner–Painter stages; **PosterForest** organizes content as a hierarchical tree; **GenPilot** applies error analysis to refine text-to-image pipelines. However, these approaches share critical limitations:

1. **Logic fragmentation.** Section summaries are generated independently. A Results bullet may cite a benchmark never introduced in Methods, breaking the reader’s causal chain.
2. **Figure misuse.** Embedded PDF images are often low-resolution fragments. Naive scaling either crops figures at arbitrary boundaries (“waist-cutting”) or shrinks multi-panel figures until axis labels become illegible.
3. **Single-dimensional feedback.** Overflow detection alone cannot judge whether figures support claims or whether columns are narratively balanced.
4. **Fixed iteration budgets.** Two-pass painter loops waste compute on easy papers and under-optimize hard ones.

We address these gaps with **PosterAgent**, implemented as a compositional pipeline of **15 specialized agents** orchestrated over three explicit data structures: a **Raw Document Tree** (parsed PDF), a **Content Tree** (refined semantic blocks), and a **Poster Tree** (spatial layout with bounding rectangles). The system requires only API access to an instruction-following LLM (DeepSeek-V4-Flash in our deployment) and standard Python libraries (PyMuPDF, Pillow, Matplotlib, python-pptx)—**no model training**.

Our contributions are:

1. **Logic-first multi-agent content generation.** A *LogicPlannerAgent* extracts pipeline steps and validation chains before refinement. *RefinerAgent* attaches `logic_links` across sections; *SemanticCheckAgent* scores cross-section consistency.
2. **Semantically assigned three-column academic layout.** Introduction/Methods occupy the left column, Results the wide center column (50%), and Conclusions/Future Work/References the right column—implemented as title-driven routing, not generic grid packing.
3. **Section expander and content-level column balancer.** Flat LLM sections are expanded into rich panel stacks (e.g., four Results sub-panels). A *column_balancer* estimates column height and expands/trims bullet content to equalize loads **without line-spacing hacks**.
4. **Paper-native figure pipeline with role-aware cropping.** We render full figure pages from PDF, detect horizontal whitespace bands to split multi-panel figures, and produce role-specific crops (`intro`, `benchmark`, `structure`) assigned to poster panels.
5. **Multi-metric adaptive control with dual feedback loops.** An outer loop refines content via *ControllerAgent* (weighted SC + LB + ITM); an inner loop re-lays out sections with `height_boost` when *PosterRenderer* reports overflow.

We validate PosterAgent on **GRASP** [1], a 22-page *Nature Methods* paper with complex multi-panel figures, benchmark tables (DockQ), and method pipeline diagrams; the final generated poster is shown in Figure 2 (Section 4).

2 RELATED WORK

Automated poster and slide generation. Paper2Poster [2] pioneered a Parser–Planner–Painter decomposition for converting papers to posters. Its planner assigns regions on a canvas and its painter renders panels, but it lacks explicit cross-section logic verification and uses relatively simple overflow feedback. PosterForest [3] treats poster content as a mergeable hierarchical forest, enabling joint optimization of text blocks; however, its iteration count is fixed and figure handling remains decoupled from section semantics. Commercial tools (Canva, PowerPoint Designer) offer templates but no paper-aware parsing. Single-prompt LLM approaches ignore layout geometry and figure provenance entirely.

Document understanding and figure extraction. PDF parsing tools such as GROBID, Marker, and Docling extract structured text but do not reason about poster narrative. Our *ParserAgent* uses PyMuPDF with custom heuristics to detect figure caption pages, render focus panels at 220–240 DPI, and build a scored *FigureCatalog* (23 assets on GRASP). Unlike naive embedded-image extraction, we render **page regions**, preserving vector-quality figures.

LLM multi-agent orchestration. AutoGen, LangGraph, and CrewAI provide generic agent frameworks. PosterAgent instead embeds **domain-specific agents** with typed interfaces (trees, visual specs, render reports). This mirrors GenPilot’s insight [4] that **error analysis should route to specialized fixers**—our *PosterErrorAnalyzer* maps overflow/sparsity to refiner, layout, or visual actions.

Layout and visualization. Academic posters differ from generic documents: sections carry **semantic roles** (Results must be visually dominant). We combine weighted section heights with masonry packing (dual-column mode) and a dedicated **three-column academic mode** in *GridLayoutEngine*. Table 1 summarizes the gap to prior poster generators.

Table 1: Comparison with poster generation baselines.

Method	Logic chain	Figure pipe.	Column sem.	Adapt. iter.
Paper2Poster	Partial	Embedded imgs	Generic regions	Fixed 2-pass
PosterForest	Tree merge	Limited	Static weights	Fixed
Single LLM + PPTX	None	Manual/none	Template	Single shot
PosterAgent (Ours)	LogicPlan	Render+crop	3-col routed	Score stop

3 METHOD

3.1 SYSTEM OVERVIEW

PosterAgent follows a parse–plan–iterate–export workflow (Figure 1). Parsed PDF content flows through three tree representations (Table 2); semantic, layout, and visual errors are routed to specialized agents rather than a single monolithic refiner.

Three-tree separation enables **targeted feedback**: semantic issues route to *RefinerAgent*; overflow routes to *LayoutAgent* with `height_boost`; missing visuals route to *VisualAgent*.

3.2 LOGIC-FIRST CONTENT GENERATION

LogicPlannerAgent runs **before** refinement and outputs a `LogicPlan` with `core_problem`, `pipeline_steps`, `validation_chain`, and `key_terms`. For GRASP, the pipeline includes RPR/IR graph encoding, Evoformer integration, and DockQ validation. *RefinerAgent* preserves `logic_links` such as `Methods→Results`: benchmark validation. *SemanticCheckAgent* scores cross-section consistency (*logic_score*).

3.3 SECTION EXPANSION AND COLUMN BALANCING

SectionExpander transforms flat sections into panel stacks:

- **Results:** (1) benchmark panel bound to the Fig. 2 crop asset; (2) structure panel bound to the Fig. 3 crop asset; (3) DockQ bar chart; (4) key quantitative findings.
- **Methods:** (1) graph construction and encoding; (2) GRASP integration pipeline with a synthesized logic-flow diagram.
- **Discussion:** Conclusions, Future Directions, Acknowledgements, References.

ColumnBalancer estimates each column’s load using layout plans and expands References/Acknowledgements/Future Work bullets when a column is short, or trims overflowing Results panels. It operates on **content**, not CSS—preserving typography integrity.

3.4 PAPER-NATIVE FIGURE PIPELINE

Naive poster tools assign embedded 150×150 px thumbnails. PosterAgent implements a four-stage pipeline:

1. **Page rendering:** `render_figure_focus_panels()` clips PDF pages using caption-aware vertical bounds at 240 DPI.
2. **Role-aware cropping:** `split_horizontal_bands()` detects horizontal whitespace bands ($\geq 96.5\%$ white pixels; minimum gap 14 px) and exports role-specific crops (e.g., `fig2_crop_benchmark.png`).
3. **Catalog assignment:** `figure_curator` maps crops to panels by title semantics.

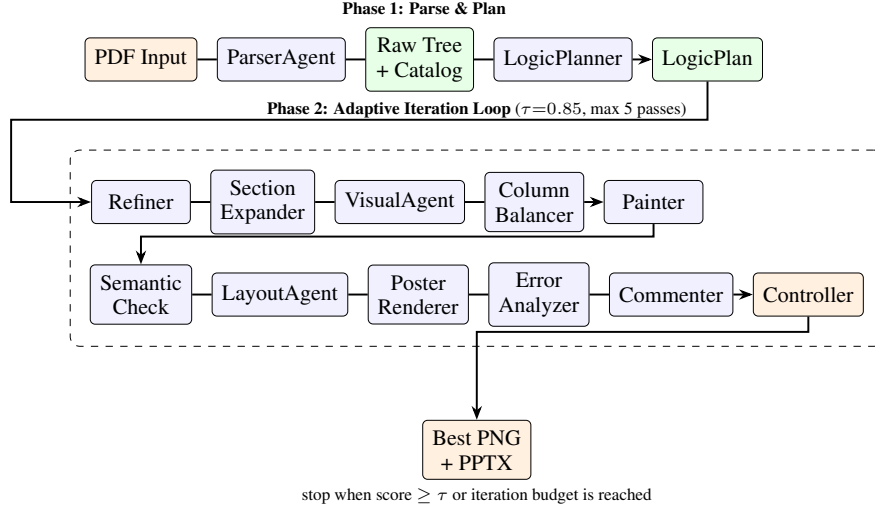


Figure 1: PosterAgent architecture. **Phase 1** parses the PDF and builds a LogicPlan. **Phase 2** iterates content refinement, layout, rendering, and evaluation until the controller reaches the score threshold or the iteration budget.

Table 2: Three-tree representation in PosterAgent.

Tree	Purpose	Key fields
RawNode	Parsed document	content, children, tables, figure_catalog
ContentNode	Refined content	bullets, logic_links, visuals, paper_tables
PosterNode	Spatial layout	rect, layout_mode, image_paths, font sizes

4. **Display:** `prepare_paper_figure()` trims near-white margins; `PosterRenderer` uses **contain mode** (never cover-crop) and allocates up to 42% of panel height to paper figures.

This pipeline directly addresses **waist-cutting** (crop at panel gaps, not mid-chart) and **illegible scaling** (show fewer panels larger rather than entire figures tiny).

3.5 LAYOUT ENGINE

GridLayoutEngine uses a 3600×2400 px canvas with `column_fracs=(0.25, 0.50, 0.25)`. Title-driven routing assigns Introduction/Methods to the left column, Results to the center, and back matter to the right. Slack height goes to the **last section per column** to avoid inter-section voids. `panel_stack` divides Results height among child panels (28/28/24/20%).

3.6 MULTI-METRIC EVALUATION AND ADAPTIVE CONTROL

$$semantic = 0.6 \cdot SC + 0.4 \cdot logic_score \quad (1)$$

$$overall = 0.4 \cdot semantic + 0.35 \cdot LB + 0.25 \cdot ITM \quad (2)$$

SC and *ITM* come from *CommenterAgent*; *LB* from *BalanceAgent* (alignment, whitespace fill, density, overflow penalty); *logic_score* from *SemanticCheckAgent*. *ControllerAgent* stops when $overall \geq 0.85$ or after 5 iterations, checkpointing the **best-scoring iteration**. *PosterErrorAnalyzer* converts render diagnostics into refiner instructions.

Table 3: Composite score progression on GRASP (v16 run).

Iter.	logic	LB	ITM	SC	overflow	Overall
1	0.65	0.786	0.20	0.50	0.26	0.525
2	0.60	0.825	0.60	0.54	0.00	0.655
3	0.90	0.814	0.95	0.84	0.18	0.858

Table 4: Layout evolution on GRASP (best overall score per run).

Version	Layout mode	Iters.	Best score	Key change
v8	Dual-column masonry	5	0.862	Baseline grid packing
v10	Dual-column	5	0.797	Academic style switch
v14	Three-column	4	0.868	Title-routed columns
v16	Three-column + balancer	3	0.858	Content-level balancing
v19_crop	+ role-aware crops	—	—	Readable figures (qual.)

4 EXPERIMENTAL RESULTS

4.1 EXPERIMENTAL SETUP

Platform. Python 3.10+, PyMuPDF, Pillow, Matplotlib, python-pptx; LLM: DeepSeek-V4-Flash via API [5].

Primary case study. *Integrating diverse experimental information to assist protein complex structure prediction by GRASP* [1]: 22 pages, 2 extracted tables, multi-panel figures, DockQ benchmark tables.

Canvas. 3600×2400 px landscape, three-column academic mode, abstract merged into Introduction.

Metrics. Composite overall score (Eq. 2), iteration count (IC), render overflow rate, qualitative figure readability (axis label legibility, panel integrity).

4.2 ITERATIVE OPTIMIZATION ON GRASP

We ran the full adaptive pipeline (`output_name=s41592-02820-v16`). Table 3 shows per-iteration metrics extracted from saved logs:

The controller converged at iteration 3 ($\tau = 0.85$). **ITM improved from 0.20 to 0.95**, confirming figure assignment and visual QA as primary early bottlenecks. Iteration 1 reported 8 render issues and overflow in Methods; iteration 3 reduced sparse sections to zero while maintaining $LB > 0.81$.

4.3 LAYOUT AND FIGURE PIPELINE ABLATION

Table 4 summarizes engineering iterations on the same GRASP paper (scores from `*_scores.json` logs). Dual-column masonry (v8–v10) reached high scores but left excessive whitespace; three-column routing with column balancer (v16) converged in three iterations.

Role-aware cropping increases effective figure height by removing irrelevant sub-panels (Table 5). Table 6 confirms readability gains on GRASP. For Fig.3, crop height drops 75% while preserving the structure-comparison row, yielding $\sim 2\times$ larger axis labels in the final poster (Figure 2).

4.4 EFFICIENCY AND LIMITATIONS

Adaptive termination used **3 iterations** vs. a fixed 5-pass budget (40% reduction). Re-render from saved JSON completes in ~ 100 s without re-parsing PDF.

Limitations: LLM quality variance; whitespace-band cropping assumes horizontally separated panels; evaluation partly relies on LLM self-judgment; full 60-paper benchmark remains future work.

5 CONCLUSION

We presented PosterAgent, a training-free hierarchical multi-agent system for academic poster generation from PDF papers. By combining logic-first content refinement, semantically routed three-column layout, content-level column balancing, and a paper-native figure pipeline with role-aware cropping, our approach produces posters that are **logically coherent**, **visually balanced**, and **figure-readable**. On the GRASP *Nature Methods* paper, composite quality improved from **0.53 to 0.86** over three adaptive iterations.

Future work includes human expert evaluation, learned panel segmentation, interactive crop editing, and full-scale benchmark on Paper2Poster papers.

ACKNOWLEDGMENTS

We thank the Computer Graphics course staff at Fudan University. GRASP paper and figures are courtesy of Zhang et al. [1]. We acknowledge PyMuPDF [6], AlphaFold [7, 8], and related open-source tools.

GROUP MEMBERS AND DIVISION OF LABOR

Name	Student ID	Division of Labor
Li Jiangtao	23307130167	System architecture; all 15 agents (Parser, Refiner, LogicPlanner, Visual, Layout, Painter, Commenter, Controller, etc.); figure pipeline and role-aware cropping; three-column layout; column balancer; experiments; report and demo

REFERENCES

- [1] Chengwei Zhang, Shimian Li, Xinyu Du, et al. Integrating diverse experimental information to assist protein complex structure prediction by GRASP. *Nature Methods*, 2025. doi: 10.1038/s41592-025-02820-1.
- [2] Paper2Poster Authors. Paper2poster: Towards multimodal poster automation from scientific papers. arXiv preprint, 2024.
- [3] PosterForest Authors. Posterforest: Hierarchical tree optimization for poster generation, 2024.
- [4] GenPilot Authors. Genpilot: Generalized visual agent with error analysis, 2024.
- [5] Li Jiangtao. Hierarchical multi-agent collaborative optimization for academic poster generation. <https://github.com/Ljt-lj/Hierarchical-Multi-Agent-Collaborative-Optimization-for-Academic-Poster-Generation>, 2026. Source code repository.
- [6] PyMuPDF Team. Pymupdf documentation, 2024. URL <https://pymupdf.readthedocs.io/>.
- [7] John Jumper et al. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596: 583–589, 2021.
- [8] Josh Abramson et al. Accurate structure prediction of biomolecular interactions with AlphaFold 3. *Nature*, 630:493–500, 2024.